

LM-GRASP: Instance-Specific Language Models for Combinatorial Construction via Online Imitation Learning

Mohand Mezmaz
SnT, University of Luxembourg
mohand.mezmaz@uni.lu

Grégoire Danoy
SnT, University of Luxembourg
gregoire.danoy@uni.lu

Abstract

Machine learning has increasingly been integrated into combinatorial optimization, most commonly through neural constructors trained via reinforcement learning on large offline datasets for a fixed problem class. Such approaches incur substantial pretraining costs and often generalize poorly outside their training distribution. We propose a different paradigm: rather than learning a constructor offline, we introduce a metaheuristic framework that reformulates the randomized constructive phase of the Greedy Randomized Adaptive Search Procedure (GRASP) as an online imitation learning task, learned from scratch on each problem instance. Under our paradigm, a local search procedure serves as an expert oracle, while a decoder-only Transformer network acts as the constructive policy. Classical GRASP variants rely on static, hand-crafted heuristic rules that are inherently myopic, ranking candidates via a localized scalar cost and unable to exploit structural regularities discovered elsewhere during the search. In contrast, our approach is entirely data-driven. The sequential construction policy emerges directly from high-quality solutions discovered during the optimization process, requiring no explicit encoding of problem-specific features.

We instantiate our methodology as **LM-GRASP**, a hybrid metaheuristic operating via an iterative learn–infer–improve cycle. Our framework trains the autoregressive policy online via behavioral cloning on a dynamic archive of elite solution trajectories, using no external data or offline pretraining. The underlying pipeline interfaces with the combinatorial domain solely through the objective evaluator used by the local search. Consequently, the system requires no domain-specific heuristic design.

We evaluate LM-GRASP on the Permutation Flow-Shop Scheduling Problem (PFSP) under the makespan minimization objective. We test our model using the standard Taillard benchmark suite (**ta51–ta60**). The selected block is highly challenging because half of the optimal solutions remain unknown, making the configuration the most discriminating setup for approximate methods. On identical hardware, LM-GRASP outperforms GPU-GRASP by 28.4 makespan units on average. Our algorithmic gain is similar in magnitude to the improvement brought by GPU acceleration over sequential execution (27.2 units), though with overlapping standard deviations. These results suggest that instance-specific, online-trained language models represent a promising and practical alternative to hand-engineered constructors in metaheuristic search. The performance edge holds particularly true for combinatorial problems whose solution landscape resists classical greedy construction.

1 Introduction

Combinatorial optimization problems arise in a broad range of scientific and industrial domains, including scheduling, logistics, network design, and resource allocation. These problems are characterized by a discrete, exponentially large solution space, a trait that renders exact methods intractable for large-scale instances. Metaheuristics such as the Greedy Randomized Adaptive Search Procedure (GRASP) [6] are therefore widely employed to obtain high-quality approximate solutions within a bounded computational budget. Over the years, numerous GRASP variants and hybrids have been proposed to improve upon the basic scheme, including path-relinking and parallel hybridizations that combine GRASP with other metaheuristic components [21], and hyper-heuristic frameworks that coordinate large pools of hand-tuned construction configurations [1]. These hybrids, however, remain fundamentally reliant on hand-engineered construction or selection rules, leaving the core limitation of static, human-designed heuristics unaddressed.

In parallel, machine learning has increasingly been integrated into metaheuristic and combinatorial optimization pipelines, from neural constructors trained via reinforcement learning for routing problems [14, 15, 3] to large language models used as configuration agents or heuristic-code generators for metaheuristics [17, 16]. These approaches demonstrate the potential of learned components to reduce the manual engineering burden of metaheuristic design, but they typically require costly offline pretraining on large problem-class datasets, or rely on general-purpose models that are not adapted to the specific instance being solved. The integration of language models directly into the search loop of a classical metaheuristic such as GRASP, trained online and without external data, remains comparatively underexplored.

A central component of GRASP is its randomized constructive phase, in which a feasible solution is built incrementally. At each step, a candidate element is drawn stochastically from a Restricted Candidate List (RCL). The generated list contains the most promising options evaluated according to a local cost metric. The resulting construction rule is inherently *human-engineered*. It encodes domain knowledge explicitly, demands extensive problem-specific tuning, and remains invariant throughout the entire search horizon. Optimization efficacy is constrained by the myopic nature of the greedy criterion. The core greedy metric evaluates decisions through a localized snapshot of the partial sequence, failing to exploit global structural patterns. Efficacy is also limited by the quality ceiling imposed by the hand-crafted cost function itself.

To address these limitations, we propose a different paradigm. Rather than engineering a construction rule a priori, we *learn* the rule from high-quality solutions discovered during the optimization process. Our framework reformulates the traditional constructive phase as an online imitation learning task [20]. In our mapping, the local search procedure acts as an inductive *expert oracle* that refines stochastically generated candidates into local optima. A decoder-only Transformer serves as the generative *parametric policy*. Finally, an active archive of the refined solutions forms the *demonstration corpus* of expert-endorsed trajectories.

A key operational property of our framework is that the sequential construction policy requires *no explicit encoding of problem topology*. The pipeline interfaces with the combinatorial domain solely through the objective function evaluator used by the local search. As a result, the constructive logic emerges entirely from data-driven sequence modeling. By framing combinatorial construction as sequential token generation, decoder-only Transformer architectures can capture non-myopic dependencies across the partial solution. The network conditions each constructive decision on the full history of preceding choices. This approach contrasts with classical greedy criteria that rely strictly on local scalar metrics. The details of the policy mapping and the iterative optimization loop are formalized in Section 4.

We instantiate our methodology as **LM-GRASP**, a hybrid metaheuristic that replaces the hand-crafted constructive phase of GRASP with an instance-specific, online-trained language model. The policy is optimized via behavioral cloning on the elite solutions accumulated during search, and the parameters are iteratively retrained as the archive evolves. LM-GRASP is not proposed as a universal alternative to GRASP. Instead, our framework serves as a targeted complement suited to problem regimes of sufficient complexity, where the overhead of online policy training is offset by the quality gains of a learned constructor.

We evaluate LM-GRASP on the Permutation Flow-Shop Scheduling Problem (PFSP) under the makespan minimization objective using the Taillard benchmark suite (**ta51–ta60**). The 50×20 block is the first configuration in the benchmark for which half of the absolute optima remain unknown, positioning our experiments at the genuine complexity frontier where the structural advantage of a learned policy over a classical greedy heuristic can be most directly assessed. Under a fixed 5-hour wall-clock budget, LM-GRASP consistently surpasses both the sequential CPU-GRASP and GPU-GRASP baselines. Our three-tiered hardware setup decouples two performance vectors: an algorithmic learning delta of 28.4 units (GPU-GRASP vs. LM-GRASP) and a hardware acceleration delta of 27.2 units (CPU-GRASP vs. GPU-GRASP). The two deltas are of comparable magnitude, suggesting that a data-driven autoregressive policy can yield optimization gains on par with porting the classical heuristic to a massively parallelized GPU.

The primary contributions of this work are as follows:

- **A Paradigm Reformulation (§4):** We present a formal framework mapping greedy randomized combinatorial construction to an online imitation learning task, with local search as an inductive expert oracle and a decoder-only Transformer as the constructive policy.
- **The LM-GRASP Framework (§4):** We develop an instance-specific, data-driven hybrid metaheuristic that replaces hand-engineered constructive rules with a policy learned entirely online. Our approach requires no external training data or offline pretraining.

- **Algorithmic vs. Hardware Quantification (§5):** We provide empirical evidence on Taillard benchmarks that LM-GRASP consistently outperforms both baselines. We also isolate the algorithmic contribution from the hardware contribution via a controlled three-tiered experimental design.
- **Instance-Complexity Positioning (§2, §5):** We characterize the complexity frontier of the Taillard benchmark and identify the 50×20 block as the principled evaluation setting. At the selected frontier, a learned construction policy becomes competitive with, and subsequently outperforms, both classical construction and hardware acceleration.
- **Taxonomy and Future Outlook (§3, §6):** We contribute a structured taxonomy of language model-integrated optimization approaches, positioning LM-GRASP within the research landscape. Finally, we identify cross-domain validation, continuous policy updates, and hyperparameter optimization as the primary directions for future work.

The remainder of this paper is organized as follows. Section 2 establishes the theoretical background, outlines classical GRASP mechanics, and motivates the choice of the PFSP as the evaluation domain. Section 3 reviews machine learning-driven optimization and situates LM-GRASP within a structured taxonomy. Section 4 details the architectural and algorithmic design of LM-GRASP. Section 5 presents the experimental configuration and empirical results. Section 6 concludes with a summary of findings and directions for future research.

2 Background

This section establishes the theoretical and algorithmic foundations underlying LM-GRASP. We begin by formalizing the Permutation Flow-Shop Scheduling Problem (PFSP) and its classical metaheuristic solution framework, GRASP. We then introduce the two learning-based components that replace the hand-crafted constructive phase. The components consist of imitation learning via behavioral cloning, alongside autoregressive sequence generation via decoder-only Transformer architectures. We conclude by positioning LM-GRASP relative to classical GRASP and motivating the choice of the PFSP as our primary evaluation domain.

2.1 The Permutation Flow-Shop Scheduling Problem

Given a set of n jobs and m machines arranged in a fixed processing sequence, each job j must be processed on every machine i . Every job requires a strictly positive duration $p_{ij} > 0$. The operational constraints require that all machines process jobs in the same order. The rule defines the *permutation* flow-shop setting. Furthermore, preemption is prohibited, and each machine can process at most one job at a time.

A feasible solution is defined by a permutation $\pi = (\pi_1, \dots, \pi_n)$ of the n jobs. The completion time $C_i(\pi, k)$ of the job at position k on machine i satisfies the following recurrence:

$$C_1(\pi, 1) = p_{1, \pi_1}, \tag{1}$$

$$C_i(\pi, 1) = C_{i-1}(\pi, 1) + p_{i, \pi_1}, \quad \text{for } i > 1, \tag{2}$$

$$C_1(\pi, k) = C_1(\pi, k-1) + p_{1, \pi_k}, \quad \text{for } k > 1, \tag{3}$$

$$C_i(\pi, k) = \max(C_{i-1}(\pi, k), C_i(\pi, k-1)) + p_{i, \pi_k}, \quad \text{for } i > 1, k > 1. \tag{4}$$

The objective is to find the permutation π^* that minimizes the *makespan* $C_{\max}(\pi) = C_m(\pi, n)$. The target value represents the completion time of the final job on the final machine. PFSP makespan minimization is strongly NP-hard for $m \geq 3$ [7].

For empirical evaluation, we use the ten Taillard instances **ta51–ta60** [23]. Each benchmark contains $n = 50$ jobs and $m = 20$ machines. The selected instances represent a standard benchmark in the scheduling literature with established best-known makespans.

Our choice of the PFSP as the evaluation domain is motivated by two structural arguments. First, the PFSP operates over a *pure permutation space*: unlike problems such as the Knapsack Problem, where flipping a single binary variable $x_i \in \{0, 1\}$ typically preserves feasibility, any incorrect token assignment in a

permutation immediately destroys the validity of the entire sequence, making the PFSP a stringent test for autoregressive generators, which must maintain global feasibility throughout the construction trajectory.

Second, the PFSP exhibits higher *representational complexity* than other permutation problems of comparable size. In the Euclidean TSP, each city is fully described by a two-dimensional coordinate vector, a compact representation that admits efficient exact methods (a 20-city instance is solvable in minutes by the Held-Karp algorithm [10]). In the PFSP, each job is instead characterized by an m -dimensional vector of processing times, one per machine, so difficulty scales directly with m : a 20×20 instance requires days of Branch-and-Bound computation [19], whereas a TSP instance of the same job count is solved trivially. This dependence on m makes the 20-machine PFSP a demanding testbed for a language model’s capacity to capture structural dependencies among solution components.

2.2 GRASP

The Greedy Randomized Adaptive Search Procedure (GRASP) [6] is a multi-start iterative metaheuristic. Each iteration consists of two distinct phases. The process begins with a *greedy randomized construction* that produces a feasible solution, which is followed by a *local search* that refines the output to a local optimum. Algorithm 1 details the general procedure.

Algorithm 1: GRASP

Input: Cost function f , greedy parameter $\alpha \in [0, 1]$, neighborhood $\mathcal{N}$, time budget T

Output: Best solution found x^*

$x^* \leftarrow \text{NULL};$

while *elapsed time* $< T$ **do**

$x \leftarrow \text{GREEDYRANDOMIZEDCONSTRUCTION}(f, \alpha);$

$x \leftarrow \text{LOCALSEARCH}(x, \mathcal{N});$

if $f(x) < f(x^*)$ **then**

$x^* \leftarrow x$

return x^* ;

Greedy Randomized Construction: Starting from an empty partial sequence, the heuristic evaluates each available candidate element e using an incremental cost metric $c(e)$. The algorithm computes the bounds $[c_{\min}, c_{\max}]$ and builds a Restricted Candidate List:

$$\text{RCL}(\alpha) = \{e \mid c(e) \leq c_{\min} + \alpha(c_{\max} - c_{\min})\}$$

One element is sampled uniformly at random from the filtered list and appended to the sequence. The parameter α interpolates between a purely greedy heuristic ($\alpha = 0$) and a uniform random walk ($\alpha = 1$).

For the PFSP, the incremental cost of inserting job j at position k is typically the makespan $C_{\max}(\pi^k)$ of the resulting partial schedule [23]. The objective function is entirely hand-crafted. The heuristic embeds domain-specific knowledge and requires a complete redesign when applied to a different combinatorial problem.

Limitations of Classical Construction: The classical GRASP constructive phase suffers from two structural limitations. First, the process is *myopic*. Candidate ranking relies entirely on a scalar, localized incremental cost. The metric cannot exploit global structural patterns in the partial solution. Second, the constructor is *memoryless*. Each construction trajectory is generated independently. As a result, the algorithm cannot reuse structural regularities from previously discovered, high-quality solutions. Together, the limitations impose a quality ceiling that additional hardware parallelism alone cannot overcome.

2.3 Imitation Learning and Behavioral Cloning

Imitation learning [13] trains a parametric policy π_θ to replicate the sequential decision-making behavior of an expert oracle π^* . This optimization utilizes a corpus of demonstrated trajectories. **Behavioral Cloning** (BC) [20] is the simplest instantiation of the paradigm. The technique reduces imitation to supervised learning by maximizing the log-likelihood of expert actions over a dataset of state-action pairs:

$$\mathcal{L}_{\text{BC}}(\theta) = -\mathbb{E}_{(s,a) \sim \mathcal{D}} [\log \pi_\theta(a \mid s)]$$

where $\mathcal{D}$ is the demonstration dataset, s is the current state, and a is the corresponding expert action.

In our setting, states correspond to partial permutations, while actions correspond to the placement of the next job. The expert oracle is instantiated by the local search procedure. Each demonstration is therefore a locally optimal permutation, representing the expert’s response to the constructive task. The policy π_θ is trained to replicate the token-by-token construction logic implicit within the elite trajectories.

2.4 Transformer Architecture and Decoder-only Autoregressive Generation

We parameterize the constructive policy π_θ with a decoder-only Transformer [25]: an L -layer stack of *masked causal self-attention* blocks, using learned positional embeddings, that models a probability distribution over discrete sequences. Given a partial sequence $x_{1:t}$, the model predicts the next token x_{t+1} , with each position attending only to preceding tokens, so that predictions are conditioned on the full construction history rather than on a local scalar summary.

Autoregressive inference proceeds sequentially. Starting from a Begin-of-Sequence (BOS) token, the model samples $x_{t+1} \sim p_\theta(\cdot \mid x_{1:t})$ and appends the selection to the current context. The process repeats until all combinatorial constraints are satisfied or an End-of-Sequence (EOS) token is produced. Generation diversity is controlled by a softmax temperature $\tau > 0$ and top- k truncation, providing a direct mechanism to balance exploration and exploitation.

Model parameters are optimized by minimizing the causal cross-entropy loss:

$$\mathcal{L}(\theta) = -\frac{1}{n} \sum_{t=1}^n \log p_\theta(x_t \mid x_{1:t-1})$$

The objective is equivalent to the behavioral cloning loss $\mathcal{L}_{\text{BC}}$ applied directly to sequential decisions. Training the Transformer on elite local optima encourages the policy to internalize long-range structural dependencies across the solution. A myopic, scalar-driven greedy heuristic cannot replicate our architecture’s structural capacity.

3 Related Work

This section situates LM-GRASP within the landscape of machine learning for combinatorial optimization. We first provide a brief overview of learning-assisted combinatorial optimization, covering both offline neural constructors and foundation models, and imitation learning for algorithmic sub-routines. We then propose a structural taxonomy to classify language model-based solvers and position LM-GRASP within the unified matrix.

3.1 Learning-Assisted Combinatorial Optimization

The use of neural networks to assist combinatorial optimization predates the recent deep learning era, with early work exploring Hopfield networks for relaxed combinatorial decision problems [11]. The field has since grown substantially, and several recent surveys provide a comprehensive overview of learning-based approaches to combinatorial optimization, ranging from supervised and reinforcement learning constructors to the more recent integration of large language models into the optimization pipeline [4].

The intersection of machine learning and combinatorial optimization has grown rapidly since Pointer Networks [26] first demonstrated autoregressive construction of TSP solutions. Kool et al. [14] integrated the routing template with the Transformer architecture. Their study established a line of deep reinforcement learning (DRL) frameworks for routing problems. POMO [15] extended the paradigm via multiple rollouts from diverse starting nodes, achieving competitive results on the TSP and CVRP. However, reinforcement frameworks suffer from a shared limitation. A separate model must be trained for each problem size and configuration, making the offline training cost substantial.

More recently, Chin et al. [3] addressed cross-instance generalization with FM-MCVRP. The framework utilizes a foundation model pretrained on 38.1 million problem–solution pairs for the capacitated vehicle routing problem. While the platform represents a highly data-intensive instantiation of a learned constructor, the pipeline requires access to a large offline dataset. In contrast, LM-GRASP requires no external data. The training corpus is synthesized online using solutions discovered for the specific instance at hand. Across this

trajectory, the dominant pattern is a clear trade-off: stronger generalization across instances is purchased at the cost of an increasingly large and expensive offline training corpus, from single-problem-size DRL policies [14] to foundation models pretrained on tens of millions of solved instances [3].

A second, more narrowly scoped trajectory uses imitation learning to approximate specific algorithmic sub-routines rather than end-to-end solution construction – for example, branching strategies in exact solvers [8] or local rewriting operators [2]. Both lines of work train their imitation policy offline, once, on a fixed dataset of expert demonstrations collected prior to deployment. Behavioral cloning has also been studied more broadly as a general-purpose imitation learning technique [20], with known limitations such as distribution shift between the training and inference state distributions, motivating extensions such as DAgger-style trajectory aggregation [22]. LM-GRASP shares the philosophy of replacing hand-crafted logic with learned approximations. However, our architecture targets the randomized construction phase of metaheuristics rather than tree-search navigation. The system also trains exclusively on dynamically generated demonstrations instead of static offline datasets. To our knowledge, no prior work applies imitation learning *online*, with the demonstration corpus generated and refreshed continuously during the search for a single target instance, rather than collected once offline.

3.2 A Taxonomy of Language Model-Based Optimization

To organize the growing landscape of language model-integrated optimization, we propose a taxonomy along two axes, illustrated in Figure 1:

- **Training Corpus Specialization:** This axis scales from *generic* approaches (a general-purpose LLM pretrained on natural language), to *problem-class-specific* setups (a model trained offline on solutions from a fixed problem class), and finally to *instance-specific* methods (a model trained online on solutions generated strictly for the target instance).
- **Functional Role at Inference:** This axis dictates the generative output of the model. Outputs can include a *problem instance* (such as algebraic constraints for a solver), a *resolution method* (such as executable heuristic code or search configurations), or a direct *solution candidate* (such as a permutation, tour, or assignment vector).

Crossing these two axes yields twelve possible categories, of which six are populated by existing or proposed work; we describe each below, along with representative examples and brief remarks on their relative maturity.

LLM-to-Instance: The model translates a natural language problem description into a formal mathematical specification (e.g., a linear program), which is then delegated to a classical solver [12]. This category is appealing as an accessibility layer over existing solvers, since it requires no change to the underlying optimization machinery, but its output quality is bounded by the correctness of the generated formalization rather than by any search capability of the model itself.

LLM-to-Parameters: The model acts as a configuration agent, tuning hyperparameters such as population size, mutation rate, or crossover rate without modifying the underlying search logic [17]. This is among the lowest-risk integrations of LLMs into metaheuristics, since the model never directly touches solution quality, only the algorithm’s operating regime; it is correspondingly one of the more mature and widely explored categories.

LLM-to-Method: The model synthesizes executable heuristic code or algorithmic structures that are compiled and run [17, 16]. Liu et al. [16] represent a prominent instantiation of this category, evolving heuristic source code through LLM-guided mutation. This category effectively delegates algorithm design itself to the language model, and has rapidly become one of the most active areas of LLM-for-optimization research, owing to its compatibility with general-purpose, off-the-shelf LLMs.

LLM-to-Solution: The model directly outputs a solution candidate. Elhenawy et al. [5] demonstrate the methodology for the TSP, prompting a general-purpose multimodal LLM with visual and textual instance representations, while Zhang et al. [29] document the underlying scalability limitations of small-scale, general-purpose LLMs used as direct numerical optimizers. This category is, to date, the least reliable in the taxonomy: because the model receives no problem-specific training signal, solution quality degrades sharply as instance size grows, and feasibility itself is not guaranteed.

PLM-to-Solution: A problem-class-specific model trained offline on domain-specific data [14, 15, 3]. The targeted systems achieve strong inference performance but carry high pretraining costs and generalize poorly across different problem classes. This is the most densely populated category in the taxonomy and the one with the longest research history, tracing back to Pointer Networks [26]; it is also the category for which empirical performance is best documented, owing to its maturity and the relative ease of benchmarking a fixed, pretrained policy.

ILM-to-Solution: An instance-specific model trained online to generate solution candidates directly. **LM-GRASP occupies the designated cell.** The Transformer is trained from scratch on elite solutions discovered for the active instance, requiring no external data or pretraining. To our knowledge, this is the first instantiation of this category: while instance-specific *conditioning* has been explored within problem-class-specific architectures (e.g., fine-tuning a pretrained policy on a single instance), no prior approach trains a generative solution constructor entirely from scratch, online, using only solutions discovered for the target instance, without recourse to any external dataset or pretrained checkpoint.

		Information		
		General		Specific
Inference outputs	Training inputs	Large corpus	Problem-related corpus	Instance-related corpus
		Large Language Model (LLM)	Problem-specialized Language Model (PLM)	Instance-specialized Language Model (ILM)
	Instance	LLM-to-Instance		
	Hyper-Parameters	LLM-to-Parameters		
	Resolution Method	LLM-to-Method		
	Solutions	LLM-to-Solution	PLM-to-Solution	ILM-to-Solution

Figure 1: Taxonomy of language model-based optimization approaches along two axes: training corpus specialization (columns: generic LM, problem-class PLM, instance-specific ILM) and functional role at inference (rows: instance, parameters, method, solution). The cell highlighted in green (ILM-to-Solution) denotes the configuration introduced in this work: to our knowledge, LM-GRASP is the first method to occupy this cell, training a generative constructor entirely online and from scratch on instance-specific demonstrations, with no external data or offline pretraining.

The gap addressed by LM-GRASP: The taxonomy above reveals a structural gap in the existing landscape. Methods that achieve strong, well-documented solution quality (PLM-to-Solution) do so only by amortizing a substantial offline training cost over a fixed problem class, and their performance degrades when applied outside the distribution of instances seen during pretraining. Conversely, methods that require no offline cost at all (LLM-to-Solution) rely on the zero-shot generalization of general-purpose language models, which is known to degrade sharply on larger or more structurally demanding instances [29]. Neither category offers a model that is simultaneously trained specifically for the instance at hand and free of external data requirements. LM-GRASP is designed to occupy precisely this intermediate position, trading the cross-instance generalization of a pretrained policy for a learned constructor specialized, at no offline cost, to the particular instance under optimization.

4 The LM-GRASP Framework

This section describes the architecture and operational lifecycle of LM-GRASP. We first provide a high-level overview and formalize the algorithmic loop. Next, we detail Phase 1 (the bootstrap phase) and Phase 2 (the iterative learn–infer–improve cycle). We then specify the algorithmic role of the generative policy and the archive. We conclude with a methodological discussion of computational trade-offs.

4.1 Algorithmic Architecture Overview

LM-GRASP replaces the hand-crafted randomized constructive phase of GRASP with a learned parametric policy. The generation routine is parameterized by a sequence model trained in an online imitation learning paradigm: the generative policy is fit to the high-quality solutions stored in the elite archive. No domain-specific heuristic rules or algebraic bounds are encoded into the constructor. Instead, the pipeline interfaces with the combinatorial domain solely through the objective function evaluator.

The optimization runtime consists of two distinct phases. Execution begins with an initial bootstrap that seeds the elite archive (Line 1). This collection is followed by an iterative learn–infer–improve loop (Lines 2–7) that runs until the computational budget is completely exhausted. The lifecycle is formalized in Algorithm 2 and illustrated in Figure 2.

Algorithm 2: LM-GRASP Execution Loop

Input: Objective evaluator f , archive capacity N_{arch} , generative sample size M , wall-clock budget T
Output: Best solution found x^*

```

1  $\mathcal{A} \leftarrow \text{BOOTSTRAP}(N_{\text{arch}});$ 
2 while elapsed time  $< T$  do
3    $\theta \leftarrow \text{TRAIN}(\pi_{\theta}, \mathcal{A});$                                 // Step (a): Online Policy Optimization
4    $\mathcal{S} \leftarrow \text{INFER}(\pi_{\theta}, M);$                             // Step (b): Autoregressive Inference
5    $\mathcal{S} \leftarrow \text{VALIDATE}(\mathcal{S});$                                // Step (c): Feasibility & Diversity Filtering
6    $\mathcal{A} \leftarrow \text{UPDATEARCHIVE}(\mathcal{A}, \mathcal{S}, N_{\text{arch}});$         // Step (d): Archive Eviction
7 return  $\arg \min_{x \in \mathcal{A}} f(x);$ 
```

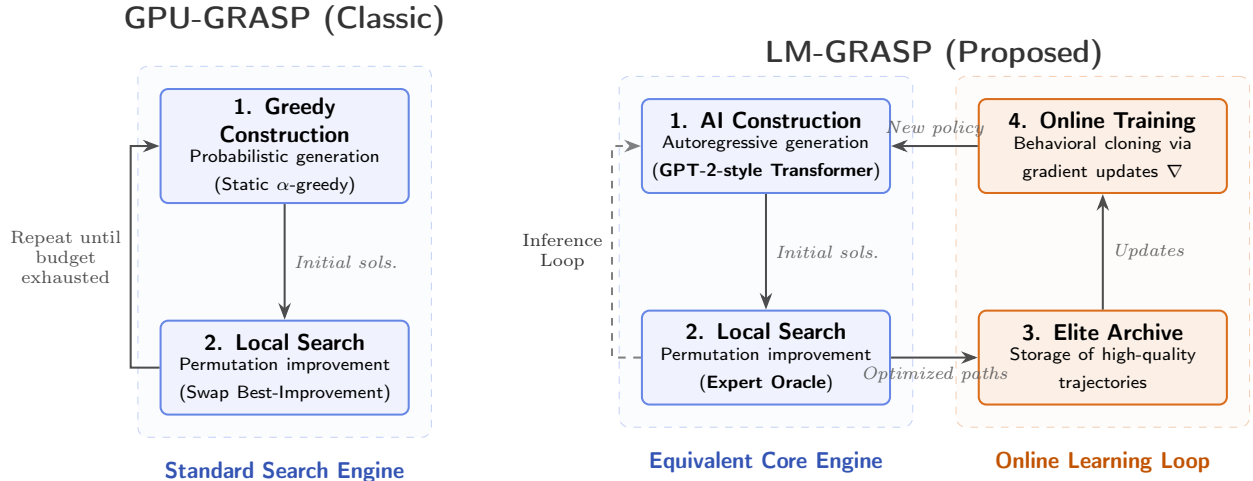


Figure 2: Structural comparison between classic GPU-GRASP and the proposed LM-GRASP framework. The addition of the third column highlights the autonomous online learning loop, which interfaces seamlessly with the equivalent local search core.

4.2 Phase 1: Prior-Free Bootstrap Routine

Before the policy can be trained, the elite archive $\mathcal{A}$ must be seeded with an initial solution corpus (Line 1 of Algorithm 2). This step is achieved by generating N_{boot} permutations via Fisher-Yates shuffles. Each sequence is then evaluated with the objective function f and inserted into the archive. No problem-specific knowledge is used at this stage. The resulting corpus provides the initial training manifold for the first policy optimization step.

4.3 Phase 2: The Iterative Learn–Infer–Improve Loop

Each generation g of the iterative phase executes the four steps of the **while** loop (Lines 2–7 of Algorithm 2) in sequence:

(a) Online Policy Optimization (Line 3): The policy π_θ is trained on the current contents of the archive $\mathcal{A}$. Because the archive enforces a hard capacity of N_{arch} solutions with fitness-based retention, the repository serves directly as the behavioral cloning corpus without additional preprocessing. Parameters are optimized by minimizing the causal cross-entropy loss:

$$\mathcal{L}(\theta) = -\frac{1}{|\mathcal{A}|} \sum_{x \in \mathcal{A}} \sum_{t=1}^n \log p_\theta(x_t \mid x_{1:t-1})$$

At generation 1, the network weights are initialized randomly. From generation 2 onward, training warm-starts from the checkpoint of the previous generation. The warm-start strategy accelerates convergence on the incrementally updated archive.

(b) Parallelized Autoregressive Inference (Line 4): The updated policy synthesizes a batch of M candidate permutations via ancestral sampling. Exploration is controlled by a top- k truncation filter with $k = \lfloor \alpha \cdot n \rfloor$ and a softmax temperature $\tau > 0$. A higher τ flattens the output distribution to encourage diversity, while a lower τ concentrates probability mass near the modes of the learned distribution.

(c) Feasibility and Diversity Filtering (Line 5): Each generated sequence is verified to be a valid, non-repeating permutation of $\{0, \dots, n-1\}$. Sequences that are invalid or already present in the archive or current inference batch are immediately discarded. The filter prevents premature mode collapse by ensuring that only structurally distinct candidates move forward to the archive update step.

(d) Archive Eviction (Line 6): The unique, valid candidates $\mathcal{S}$ are inserted into the archive. If the resulting size exceeds N_{arch} , the highest-cost entries are removed. This eviction rule maintains a direct correspondence between archive size and training corpus size. Consequently, the updated archive can be used immediately in the next generation without downsampling.

4.4 Generative Policy and Archive: Algorithmic Roles

The policy π_θ is an autoregressive sequence model whose generative process is mapped onto the combinatorial construction step of GRASP as follows:

- A **token** corresponds directly to a **solution component** (such as a job index in a scheduling permutation).
- A **complete sequence** corresponds to a **feasible solution**.
- **Token selection** via the softmax distribution corresponds to the **stochastic element selection** from an RCL.
- **Sequence likelihood** serves as an implicit proxy for **solution quality**. Maximizing token log-likelihood encourages the policy to reproduce the structural patterns of elite solutions.

The vocabulary consists of the n job indices augmented with Begin-of-Sequence (BOS) and End-of-Sequence (EOS) tokens, giving a vocabulary size and context window of $n + 2$.

The archive $\mathcal{A}$ plays a dual algorithmic role: it is both the elite pool from which the next bootstrap of training data is drawn (Line 3) and the receptacle into which newly validated candidates are written (Line 6).

Telemetry collected at each generation tracks counts of generated, invalid, duplicate, and novel solutions, as well as the min, mean, and max objective values at three checkpoints per iteration: before inference, after inference, and after the archive update.

4.5 Methodological Discussion

The key difference between LM-GRASP and classical GRASP variants lies in how the computational budget is allocated. GPU-GRASP maximizes the number of independent random constructions per unit time via massive hardware parallelism. In contrast, LM-GRASP invests part of the budget into policy training (Line 3). This mechanism biases subsequent sampling (Line 4) toward high-quality regions of the search space. We accept higher per-iteration overhead in exchange for more focused candidate generation.

5 Experiments

This section presents an empirical evaluation of LM-GRASP against classical metaheuristic baselines. We describe the experimental setup, report quantitative results on the Taillard benchmark suite, and analyze convergence dynamics alongside intra-iteration policy behavior.

5.1 Implementation Details

This subsection describes concrete implementation choices used in our experiments; these are not part of the algorithm’s specification and can be substituted without altering Algorithm 2.

Policy network: The policy π_θ is implemented as a GPT-2-style decoder-only Transformer in PyTorch, with L causally-masked self-attention layers, pre-LayerNorm blocks, learned positional embeddings, and weight tying between the token embedding matrix and the output projection head.

Training procedure: Training uses the AdamW optimizer. Weight decay is applied to parameter tensors of dimension ≥ 2 , and the schedule combines cosine decay with linear warmup alongside gradient norm clipping. An early-stopping monitor tracks validation loss and terminates training if the improvement fails to exceed a threshold ε for P consecutive evaluations, conserving the wall-clock budget. Inference (Line 4) is vectorized into batched tensor operations of size B_{inf} .

Archive storage: The elite archive $\mathcal{A}$ is backed by a SQLite database. Permutations are stored as text strings serving as unique primary keys, paired with their objective cost; this layout prevents duplicate entries and supports efficient SQL queries for training-corpus extraction, fitness-distribution queries, and bounded eviction (Line 6). Bootstrap solutions (Line 1) are written directly to this database. A dedicated `inference_stats` table records the per-generation telemetry described above.

5.2 Experimental Setup

Computational Infrastructure

We executed our experiments on two distinct platforms within the university cluster:

- **Sequential CPU Baseline (Aion Cluster):** A BullSequana X2410 node equipped with two 64-core AMD EPYC 7H12 processors running at 2.6 GHz. The configuration provides 128 physical cores total alongside 256 GB of DDR4 RAM running at 3200 MT/s.
- **GPU-Accelerated Frameworks (Iris Cluster):** A Dell PowerEdge C4140 node featuring dual 14-core Intel Xeon Gold 6132 CPUs at 2.6 GHz. The platform includes 768 GB of DDR4 RAM at 2666 MT/s, paired with four NVIDIA Tesla V100-SXM2 GPUs interconnected via NVLink (300 GB/s). Both GPU-GRASP and LM-GRASP were bound to a single isolated V100 instance to eliminate device-sharing interference.

Benchmark Protocol

We evaluated all methods on the ten Taillard PFSP instances **ta51–ta60** [23]. The problem set features $n = 50$ jobs and $m = 20$ machines ($Fm||C_{\max}$), with a collective average best-known makespan of 3708. Each method was run 10 times per instance under an identical 5-hour wall-clock budget.

Choice of Instance Block: The Taillard benchmark spans configurations ranging from 20×5 to 500×20 . Table 1 reports the number of instances for which the exact optimum is known across the full range as of June 2026, revealing that combinatorial difficulty is not monotone in instance size: very large configurations such as 500×20 or 200×20 are comparatively tractable, as their fitness landscapes become highly homogeneous at scale, whereas the genuine complexity frontier concentrates at the intermediate 50×20 and 100×20 blocks, where the search landscape is maximally rugged and a significant fraction of optima remain unknown.

The 50×20 block (**ta51–ta60**) is the first configuration in the benchmark where exactly half of the absolute optima are still open, making it the natural threshold at which the structural advantage of a learned construction policy over a classical greedy heuristic can be directly assessed: below this scale, the computational efficiency of classical GRASP is unmatched, while at and above it, the rugged search topology motivates the structural awareness conferred by the Transformer policy.

Table 1: Number of Taillard instances for which the exact optimum is known (updated: June 2026). Configurations sharing the same count are grouped; the two complexity-frontier blocks are shown individually.

Configuration ($n \times m$)	Instance range	Known optima / 10
20×5 , 20×10 , 20×20 , 50×5 , 50×10	ta001 – ta050	10 / 10 (each)
50×20	ta051 – ta060	5 / 10
100×5 , 100×10	ta061 – ta080	10 / 10 (each)
100×20	ta081 – ta090	4 / 10
200×10 , 500×20	ta091 – ta100, ta111 – ta120	10 / 10 (each)
200×20	ta101 – ta110	9 / 10
Total		108 / 120

The 50×20 Taillard block is a recognized challenge for both exact and approximate methods. Exact Branch-and-Bound requires substantial computational resources on the designated instances. For example, instance **ta56** required 25 days of computation (approximately 22 CPU-years) across 328 processors when first solved exactly by Mezmaz et al. [19]. More recently, Gmys [9] solved several previously open instances using up to 384 V100 GPUs on the Jean Zay supercomputer.

For approximate methods, the fitness landscape is highly rugged [27], and standard local search operators stagnate readily. Previous GRASP-based approaches required coordinating 315 distinct hand-tuned configurations via a parallel multi-core hyper-heuristic to remain competitive on the target block [1]. The severe difficulty motivates the use of a learned construction policy as an alternative to extensive manual tuning.

Evaluated Methods

- **CPU-GRASP:** Standard GRASP utilizing a hand-crafted constructive heuristic. The solver ranks jobs by their incremental makespan contribution and runs sequentially on the Aion CPU node.
- **GPU-GRASP:** A GPU-based GRASP variant using the same hand-crafted heuristic as CPU-GRASP. The script runs on the Iris V100 GPU. The performance difference between CPU-GRASP and GPU-GRASP isolates the effect of hardware acceleration under an identical construction rule.
- **LM-GRASP:** Our proposed framework, running on the same V100 GPU as GPU-GRASP. The construction policy is a decoder-only Transformer trained online via behavioral cloning on the top- K solutions in a persistent archive. The system requires no domain-specific heuristic rules.

Model and Optimization Configuration

Architecture: Following the implementation described in Section 5.1, the Transformer policy uses $L = 14$ attention layers, 12 attention heads, and an embedding dimension of 768. We apply a dropout rate of 0.2 to the residual connections, token embeddings, and attention weights.

Training: The archive size is fixed at $K = 5,000$ elite sequences. We use a learning rate of 6×10^{-4} , a 500-step linear warmup, a weight decay of 0.1, and a gradient norm clip of 1.0, with gradient accumulation over 4 steps (effective batch size 64). Training runs for at most 2,001 steps, with validation on a 10% holdout every 500 steps; early stopping triggers if validation loss does not improve by more than 10^{-4} over 15 consecutive evaluations.

Inference: Generation uses a softmax temperature of $\tau = 1.3$ and top- k filtering with $k = \lfloor 0.2 \times n \rfloor$, with candidates sampled in batches of 512 ($B_{\text{inf}} = 512$), targeting a total pool of $M = 10,000$ per generation.

Hyperparameter Selection: We established the architectural and optimization hyperparameters reported above empirically through a limited set of preliminary trials. We did not use exhaustive search procedures such as grid search or Bayesian optimization. The fact that competitive performance is achieved under an initial calibration underscores the robustness of the LM-GRASP framework. A systematic sensitivity analysis would be particularly costly for architectures of the current scale. The exploration falls outside the scope of our introductory study and is identified as a natural direction for future work.

5.3 Results and Analysis

Tables 2 and 3 report the best-found makespan across all runs alongside the per-run mean makespan.

Table 2: Minimum makespan found across all runs and total run-level wins per method on Taillard instances **ta51–ta60**. **Bold** values indicate the best performer among tested methods.

Instance	Minimum Makespan (min)			Best known	Is it Opt.?	Win Count (↑)		
	CPU-GRASP	GPU-GRASP	LM-GRASP			CPU-GRASP	GPU-GRASP	LM-GRASP
ta51	3950	3927	3898	3846	—	0	0	10
ta52	3806	3777	3751	3699	✓	0	0	10
ta53	3758	3731	3693	3640	✓	0	0	10
ta54	3823	3795	3769	3719	—	0	0	10
ta55	3719	3706	3662	3610	—	0	0	10
ta56	3775	3757	3725	3679	✓	0	0	10
ta57	3810	3784	3760	3704	✓	0	0	10
ta58	3812	3789	3753	3691	✓	0	0	10
ta59	3856	3822	3797	3741	—	0	0	10
ta60	3832	3821	3811	3755	—	0	0	10
Average / Total	3814	3790	3761	3708		0	0	100

Table 3: Average of per-run best makespans per method on Taillard instances **ta51–ta60**. Values show mean $\pm$ std. dev. over 10 runs.

Instance	#Runs	Average Makespan Per Method			Pairwise Gap Evolution	
		CPU-GRASP	GPU-GRASP	LM-GRASP	CPU/GPU	GPU/LM
ta51	10	3958.6 $\pm$ 05.8	3933.3 $\pm$ 03.7	3907.4 $\pm$ 04.1	25.1 $\pm$ 06.2	26.0 $\pm$ 04.9
ta52	10	3820.1 $\pm$ 08.4	3787.4 $\pm$ 08.0	3758.8 $\pm$ 05.0	37.3 $\pm$ 12.0	27.8 $\pm$ 10.4
ta53	10	3774.9 $\pm$ 09.1	3740.7 $\pm$ 05.9	3710.3 $\pm$ 09.7	31.1 $\pm$ 12.5	29.7 $\pm$ 12.9
ta54	10	3831.2 $\pm$ 06.6	3802.4 $\pm$ 03.8	3776.6 $\pm$ 04.5	28.0 $\pm$ 10.1	26.0 $\pm$ 06.5
ta55	10	3740.9 $\pm$ 13.4	3714.6 $\pm$ 05.5	3675.4 $\pm$ 07.3	24.4 $\pm$ 16.0	39.3 $\pm$ 10.8
ta56	10	3789.5 $\pm$ 10.1	3768.7 $\pm$ 07.2	3739.7 $\pm$ 06.6	21.3 $\pm$ 14.7	28.3 $\pm$ 07.8
ta57	10	3822.4 $\pm$ 07.2	3797.6 $\pm$ 07.1	3767.4 $\pm$ 04.0	25.0 $\pm$ 12.7	30.7 $\pm$ 08.1
ta58	10	3825.6 $\pm$ 09.9	3793.3 $\pm$ 04.3	3761.7 $\pm$ 04.8	32.4 $\pm$ 10.0	31.0 $\pm$ 08.4
ta59	10	3864.1 $\pm$ 05.5	3836.1 $\pm$ 06.2	3801.8 $\pm$ 03.2	27.4 $\pm$ 05.9	34.2 $\pm$ 06.7
ta60	10	3849.0 $\pm$ 09.1	3830.2 $\pm$ 06.5	3818.8 $\pm$ 05.0	19.6 $\pm$ 09.9	11.2 $\pm$ 07.2
All instances	100	3827.6 $\pm$ 08.5	3800.4 $\pm$ 05.8	3771.8 $\pm$ 05.4	27.2 $\pm$ 11.0	28.4 $\pm$ 08.4

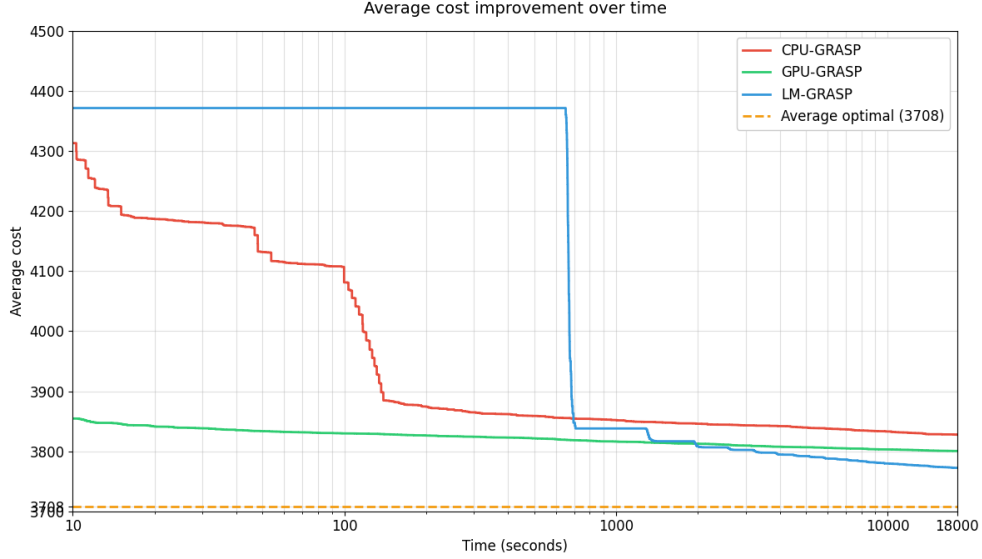


Figure 3: Average optimization trajectories over time (log scale) across **ta51–ta60** for all three methods. The dashed line marks the average best-known makespan (3708). GPU-GRASP converges to a plateau (≈ 3800.4) within the first 30 seconds. LM-GRASP spends approximately 660 seconds on bootstrapping and initial policy training, then improves steadily throughout the remaining budget, reaching a mean terminal makespan of 3771.8.

Comparative Performance: LM-GRASP achieves a mean makespan of 3771.8 ± 5.4 across all instances and runs. In comparison, GPU-GRASP scores 3800.4 ± 5.8 and CPU-GRASP scores 3827.6 ± 8.5 . LM-GRASP emerges as the best-performing method on all ten instances (Table 2) and across all 100 individual runs (Table 3).

Our three-way design decouples two performance components. The **hardware acceleration delta** (CPU-GRASP vs. GPU-GRASP) averages 27.2 units. Because both variants use the exact same construction rule, the throughput gap reflects pure parallelization capacity. The **algorithmic learning delta** (GPU-GRASP vs. LM-GRASP) averages 28.4 units. Because both variants run on identical V100 hardware, the resulting improvement is directly attributable to the learned construction policy.

The two deltas are of similar magnitude, showing a difference of only 1.2 units, with standard deviations of ± 11.0 and ± 8.4 respectively. The proximity suggests that the algorithmic contribution is at least comparable to the hardware contribution in our testing framework. We note that the comparison is specific to the evaluated instances and budget.

Convergence Dynamics: Figure 3 illustrates divergent convergence profiles: GPU-GRASP quickly exhausts its hardware-parallel exploration and plateaus, whereas LM-GRASP follows a two-phase trajectory, investing early budget in bootstrapping and policy training before improving steadily thereafter. Crucially, the LM-GRASP trajectory has not flattened at budget exhaustion, suggesting that additional compute time would yield further gains.

Intra-Iteration Policy Generalization: Figure 5 shows that, from iteration 5 onward, the policy generates solutions of lower mean makespan than its own training corpus, indicating that it does not merely reproduce archived solutions but generalizes to produce novel sequences of higher quality.

Convergence of the Generative Process: Figure 4 documents a progressive contraction in the number of archive-improving candidates across iterations. Rather than a pathological failure mode, this is the natural signature of convergence: as the search identifies and archives the highest-quality regions of the solution space, fewer unexplored improvements remain available, consistent with an optimization loop approaching the limits of the accessible search space under the current budget.

These results support our central claim: a construction policy learned online via behavioral cloning can replace a hand-crafted greedy heuristic and produce better solutions under an equivalent resource budget,

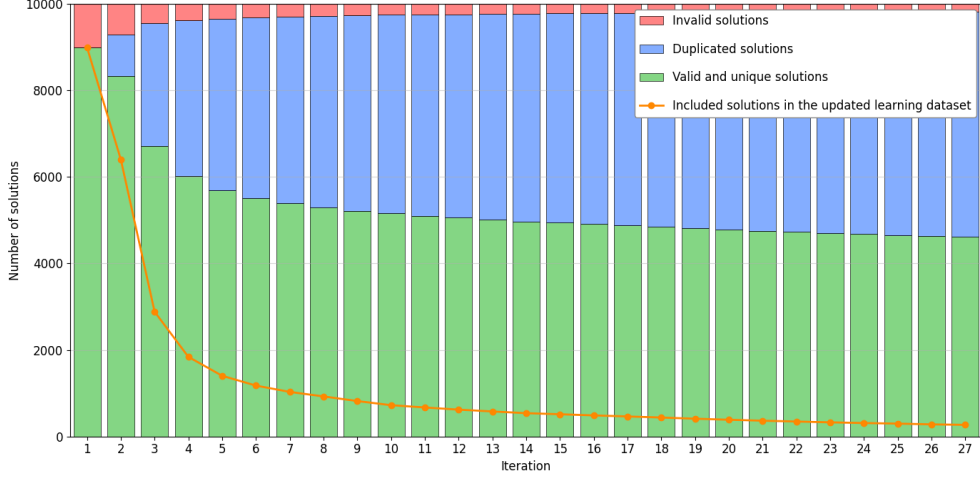


Figure 4: Breakdown of the 10,000-candidate batch by category across iterations. The proportion of novel, archive-improving candidates contracts from $\approx 8,986$ in iteration 1 to ≈ 299 by iteration 25, while the share of duplicates increases correspondingly. This contraction reflects the natural exhaustion of promising, previously unexplored regions as the search converges.

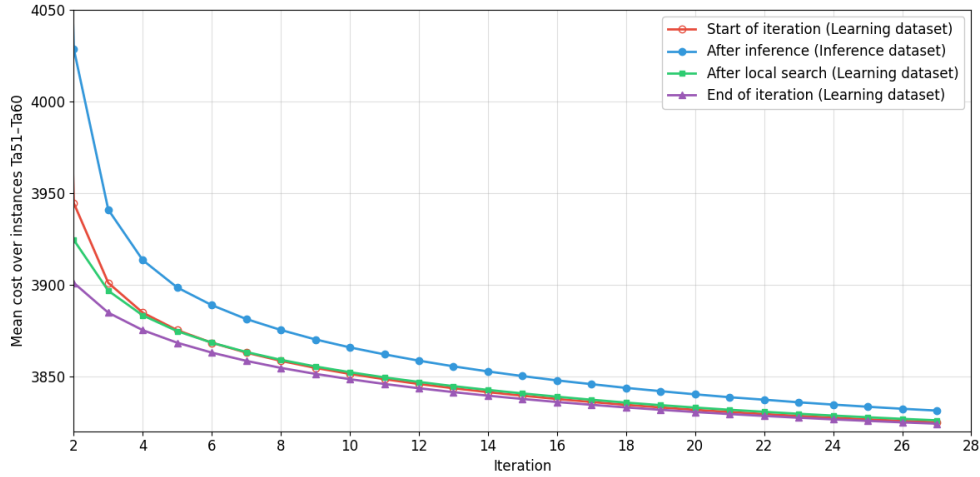


Figure 5: Intra-iteration makespan at four checkpoints per generation averaged across `ta51-ta60`: start of iteration (training corpus baseline), after inference, after local search, and end of iteration (archived). The monotonic decrease of end-of-iteration values from ≈ 3900 to ≈ 3824 confirms stable convergence of the optimization loop.

with the local search oracle alone providing a sufficient training signal and no domain-specific feature encoding. Compared to prior work on this benchmark block, LM-GRASP achieves strong performance through a single neural network, avoiding the coordinated selection among hundreds of hand-tuned configurations required by previous GRASP-based approaches [1].

The two methods represent different strategies for using a fixed budget: GPU-GRASP prioritizes breadth by maximizing the number of independent constructions, while LM-GRASP prioritizes depth by investing in a generative model that biases sampling toward high-quality regions. Because the anytime trajectories show no plateau at budget exhaustion (Figure 3), extending the search horizon – through longer budgets or continuous policy updates – represents a natural path to further improvement.

6 Conclusion

We introduced LM-GRASP, a hybrid metaheuristic that replaces the randomized constructive phase of GRASP with a policy learned online via behavioral cloning. A local search procedure serves as the expert oracle, a decoder-only Transformer serves as the constructive policy, and the training corpus is built from elite solutions discovered during the search itself, eliminating the need for domain-specific feature engineering or external training data.

On the Taillard **ta51–ta60** instances under a 5-hour budget, LM-GRASP outperforms both CPU-GRASP and GPU-GRASP on every instance and every run (Section 5). Our three-tiered hardware design isolates a hardware acceleration delta (CPU-GRASP vs. GPU-GRASP) from an algorithmic learning delta (GPU-GRASP vs. LM-GRASP); the two are of comparable magnitude, which suggests that a learned construction policy is at least as effective as hardware acceleration at improving solution quality. Intra-iteration analysis further shows that, from iteration 5 onward, the policy generates solutions of higher quality than its own training corpus, confirming that the model generalizes beyond the archived demonstrations rather than reproducing them.

The 50×20 Taillard block was chosen because it sits at the complexity frontier of the benchmark, where classical greedy construction begins to show its structural limits. Consistent with the No Free Lunch theorem [28], LM-GRASP is not intended as a universal replacement for GRASP: for smaller or lower-dimensional instances, classical GRASP remains the more efficient choice, and our tool is best viewed as a targeted alternative for problem regimes complex enough to amortize the cost of online policy training. Consistent with this reading, the anytime trajectories show no plateau at budget exhaustion, and the rate of archive-improving candidates declines gradually rather than collapsing outright, indicating that the search has not yet saturated the accessible horizon.

Several extensions follow naturally from our study:

- **Cross-Domain Validation:** The pipeline requires only an integer sequence representation paired with an objective evaluator, so it applies directly to other combinatorial problems. Evaluating LM-GRASP on the TSP, the Quadratic Assignment Problem, and bin-packing will test whether the observed advantages transfer across problem classes, and help delineate the complexity threshold below which classical GRASP remains preferable.
- **Continuous Policy Updates:** Replacing batch retraining with continuous online gradient updates as elite solutions enter the archive would reduce the latency between discovery and learning, potentially accelerating convergence and extending the effective search horizon within a fixed budget.
- **Hyperparameter Optimization:** The architectural and inference parameters used in our evaluation were set empirically through preliminary trials. A systematic sensitivity analysis, particularly of the temperature schedule and archive capacity, is a necessary next step to characterize the operating envelope of LM-GRASP.

Our work demonstrates that a construction policy learned online from self-generated expert demonstrations can replace a hand-crafted greedy heuristic and produce highly competitive solutions under an equivalent computational budget, without any problem-specific feature design.

Code Availability

The implementation of LM-GRASP is available as part of the COLM platform [18], at <https://gitlab.com/uniluxembourg/snt/pcog/colm>.

Acknowledgment

The experiments presented in this paper were carried out using the HPC facilities of the University of Luxembourg [24] – see <https://hpc.uni.lu>

References

- [1] E. Alekseeva, M. Mezmaz, D. Tuytens, and N. Melab. Parallel multi-core hyper-heuristic GRASP to solve permutation flow-shop problem. *Concurrency and Computation: Practice and Experience*, 29(9): e3835, 2017.
- [2] X. Chen and Y. Tian. Learning to perform local rewriting for combinatorial optimization. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.
- [3] S. J. Chin, M. Winkenbach, and A. Srivastava. Learning to deliver: A foundation model for the Montreal capacitated vehicle routing problem, 2024. arXiv preprint arXiv:2403.00026.
- [4] F. Da Ros, M. Soprano, L. Di Gaspero, and K. Roitero. Large language models for combinatorial optimization: A systematic review, 2025. arXiv preprint arXiv:2507.03637.
- [5] M. Elhenawy et al. Visual reasoning and multi-agent approach in multimodal large language models: Solving TSP and MTSP combinatorial challenges. *Machine Learning and Knowledge Extraction*, 6(3): 1894–1920, 2024.
- [6] T. A. Feo and M. G. C. Resende. Greedy randomized adaptive search procedures. *Journal of Global Optimization*, 6(2):109–133, 1995.
- [7] M. R. Garey, D. S. Johnson, and R. Sethi. The complexity of flowshop and jobshop scheduling. *Mathematics of Operations Research*, 1(2):117–129, 1976.
- [8] M. Gasse, D. Chételat, N. Ferroni, L. Charlin, and A. Lodi. Exact combinatorial optimization with graph convolutional neural networks. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- [9] J. Gmys. Exactly solving hard permutation flowshop scheduling problems on peta-scale GPU-accelerated supercomputers. *INFORMS Journal on Computing*, 34(5):2448–2466, 2022.
- [10] M. Held and R. M. Karp. A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics*, 10(1):196–210, 1962.
- [11] J. J. Hopfield and D. W. Tank. “Neural” computation of decisions in optimization problems. *Biological Cybernetics*, 52(3):141–152, 1985.
- [12] X. Huang, Q. Shen, Y. Hu, A. Gao, and B. Wang. LLMs for mathematical modeling: Towards bridging the gap between natural and mathematical languages. In *Findings of ACL: NAACL 2025*, pages 2678–2710, 2025.
- [13] A. Hussein, M. M. Gaber, E. Elyan, and C. Jayne. Imitation learning: A survey of learning methods. *ACM Computing Surveys*, 50(2):1–35, 2017.
- [14] W. Kool, H. van Hoof, and M. Welling. Attention, learn to solve routing problems! In *International Conference on Learning Representations (ICLR)*, 2019.

- [15] Y.-D. Kwon, J. Choo, B. Kim, I. Yoon, Y. Gwon, and S. Min. POMO: Policy optimization with multiple optima for reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 21188–21198, 2020.
- [16] F. Liu, X. Tong, M. Yuan, X. Lin, F. Luo, Z. Wang, Z. Lu, and Q. Zhang. Evolution of heuristics: Towards efficient automatic algorithm design using large language model, 2024. arXiv preprint arXiv:2401.02051.
- [17] A. Martinek, S. Lukasik, and A. H. Gandomi. Large language models as tuning agents of metaheuristics. In *ESANN 2024*, 2024.
- [18] M. Mezma. COLM: A platform for integrating language models in combinatorial optimization. <https://gitlab.com/uniluxembourg/snt/pcog/colm>, 2026.
- [19] M. Mezma, N. Melab, and E.-G. Talbi. A grid-enabled branch and bound algorithm for solving challenging combinatorial optimization problems. In *Proceedings of the 2007 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 1–9, 2007.
- [20] D. A. Pomerleau. Efficient training of artificial neural networks for autonomous navigation. *Neural Computation*, 3(1):88–97, 1991.
- [21] M. G. Ravetti, C. Riveros, A. Mendes, M. G. Resende, and P. M. Pardalos. Parallel hybrid heuristics for the permutation flow shop problem. *Annals of Operations Research*, 199(1):269–284, 2012.
- [22] S. Ross, G. Gordon, and D. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, pages 627–635. JMLR Workshop and Conference Proceedings, 2011.
- [23] E. Taillard. Benchmarks for basic scheduling problems. *European Journal of Operational Research*, 64(2):278–285, 1993.
- [24] S. Varrette, H. Cartiaux, S. Peter, E. Kieffer, T. Valette, and A. Olloh. Management of an academic HPC & research computing facility: The ULHPC experience 2.0. In *Proceedings of the 6th ACM High Performance Computing and Cluster Technologies Conference (HPCCT 2022)*, Fuzhou, China, 2022. Association for Computing Machinery (ACM). ISBN 978-1-4503-9664-6.
- [25] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [26] O. Vinyals, M. Fortunato, and N. Jaitly. Pointer networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, 2015.
- [27] J. P. Watson, J. C. Beck, A. E. Howe, and L. D. Whitley. Problem difficulty for permutation flow-shop scheduling. *Artificial Intelligence*, 149(2):199–231, 2003.
- [28] D. H. Wolpert and W. G. Macready. No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1):67–82, 2002.
- [29] T. Zhang, J. Yuan, and S. Avestimehr. Revisiting OPRO: The limitations of small-scale LLMs as optimizers. In *Findings of ACL 2024*, pages 1727–1735, 2024.